

ASSEMBLY LANGUAGE

- Syntax of the Basic Computer assembly language
 - Each line is arranged in three columns called fields
 - *Label* field
 - May be empty or may specify a symbolic address consists of up to 3 characters
 - Terminated by a comma
 - *Instruction* field
 - Specifies a machine or a pseudo instruction
 - May specify one of
 - * Memory reference instr. (MRI)
 - MRI consists of two or three symbols separated by spaces.
 - ADD OPR (direct address MRI)
 - ADD PTR I (indirect address MRI)
 - * Register reference or input-output instr.
 - Non-MRI does not have an address part
 - * Pseudo instruction with or without an operand
 - Symbolic address used in the instruction field must be defined somewhere as a label
 - *Comment* field
 - May be empty or may include a comment



PSEUDO-INSTRUCTIONS

ORG N

Hexadecimal number N is the memory location
for the instruction or operand listed in the following line

END

Denotes the end of symbolic program

DEC N

Signed decimal number N to be converted to the binary

HEX N

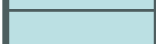
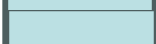
Hexadecimal number N to be converted to the binary

Example: Assembly language program to subtract two numbers

	ORG 100	/ Origin of program is location 100
	LDA SUB	/ Load subtrahend to AC
	CMA	/ Complement AC
	INC	/ Increment AC
	ADD MIN	/ Add minuend to AC
	STA DIF	/ Store difference
	HLT	/ Halt computer
MIN,	DEC 83	/ Minuend
SUB,	DEC -23	/ Subtrahend
DIF,	HEX 0	/ Difference stored here
	END	/ End of symbolic program



TRANSLATION TO BINARY

<i>Hexadecimal Code</i>		<i>Symbolic Program</i>
<i>Location</i>	<i>Content</i>	
100		ORG 100
101		LDA SUB
102		CMA
103		INC
104		ADD MIN
105		STA DIF
106		HLT
107		MIN, SUB, DIF,
108		DEC 83 DEC -23 HEX 0 END



TRANSLATION TO BINARY

<i>Hexadecimal Code</i>		<i>Symbolic Program</i>
<i>Location</i>	<i>Content</i>	
100	2107	ORG 100
101	7200	LDA SUB
102	7020	CMA
103	1106	INC
104	3108	ADD MIN
105	7001	STA DIF
106	0053	HLT
107	FFE9	MIN, SUB, DIF,
108	0000	DEC 83 DEC -23 HEX 0 END



Representation of symbolic program in memory

- Prior to conversion into binary, assembly program should be saved in the memory.
- Loader program is used to input the assembly characters into memory.
- Assembly program in memory is in the alphanumeric (ASCII) form



Character	Code (2 digits of HEX)	Character	Code (2 digits of HEX)	Character	Code (2 digits of HEX)
A	41	Q	51	6	36
B	42	R	52	7	37
C	43	S	53	8	38
D	44	T	54	9	39
E	45	U	55	SPACE	20
F	46	V	56	(	28
G	47	X	57	)	29
H	48	X	58	*	2A
I	49	Y	59	+	2B
J	4A	Z	5A	,	2C
K	4B	0	30	-	2D
L	4C	1	31	.	2E
M	4D	2	32	/	2F
N	4E	3	33	=	3D
O	4F	4	34	CR	0D
P	50	5	35		

Assembler recognizes, **CR** code as the end of a line of code.



Representation of symbolic program in memory -- Line of code

- From the table of previous slide, it is clear that each character is represented by 2 hexadecimal digits.
- We know that each word of memory = 16 bit (4 hexadecimal digits)
- So, each word can have max. of 2 characters.
- Label is terminated by a comma
- Operation and Address symbol is terminated by space
- Code line is terminated by CR.

Example:

PL3, LDA SUB I



PL3, LDA SUB I

Memory word	Symbol	Hexadecimal code	Binary representation
1	P L	50 4C	0101 000 0100 1100
2	3 ,	33 2C	0011 0011 0010 1100
3	L D	4C 44	0100 1100 0100 0100
4	A	41 20	0100 0001 0010 0000
5	S U	53 55	0101 0011 0101 0101
6	B	42 20	0100 0010 0010 0000
7	I CR	49 0D	0100 1001 0000 1101



ASSEMBLER - FIRST PASS -

Assembler

Source Program - Symbolic Assembly Language Program

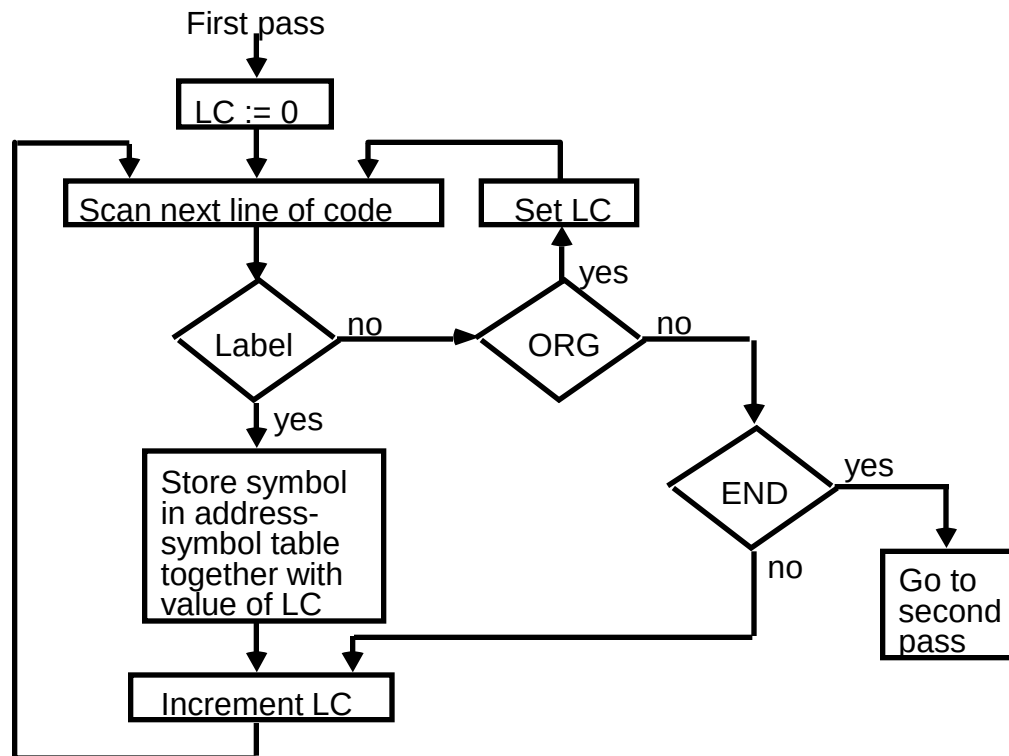
Object Program - Binary Machine Language Program

Two pass assembler

1st pass: generates a table that correlates all user defined (address) symbols with their binary equivalent value

2nd pass: binary translation

First pass

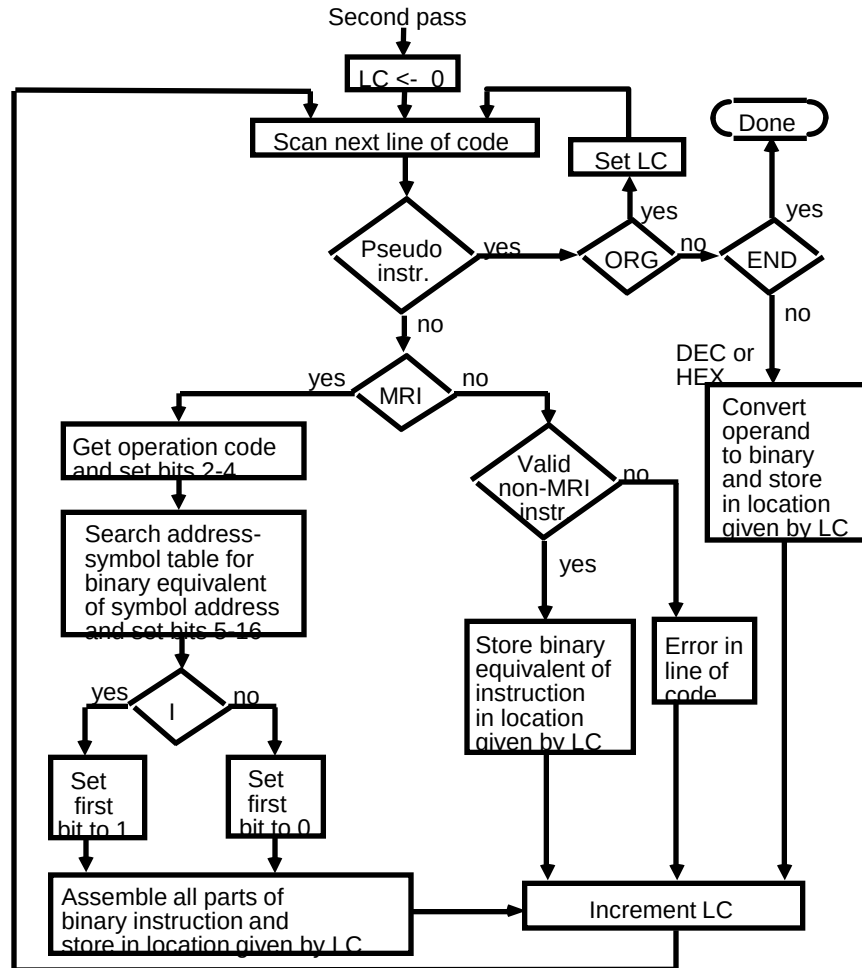


ASSEMBLER - SECOND PASS -

Second Pass

Machine instructions are translated by means of table-lookup procedures;

(1. Pseudo-Instruction Table, 2. MRI Table, 3. Non-MRI Table 4. Address Symbol Table)



PROGRAM LOOPS

Loop: A sequence of instructions that are executed many times, each with a different set of data

Fortran program to add 100 numbers:

```

DIMENSION A(100)
INTEGER SUM, A
SUM = 0
DO 3 J = 1, 100
3  SUM = SUM + A(J)

```

Assembly-language program to add 100 numbers:

	ORG 100	/ Origin of program is HEX 100
	LDA ADS	/ Load first address of operand
	STA PTR	/ Store in pointer
	LDA NBR	/ Load -100
	STA CTR	/ Store in counter
	CLA	/ Clear AC
LOP,	ADD PTR I	/ Add an operand to AC
	ISZ PTR	/ Increment pointer
	ISZ CTR	/ Increment counter
	BUN LOP	/ Repeat loop again
	STA SUM	/ Store sum
	HLT	/ Halt
ADS,	HEX 150	/ First address of operands
PTR,	HEX 0	/ Reserved for a pointer
NBR,	DEC -100	/ Initial value for a counter
CTR,	HEX 0	/ Reserved for a counter
SUM,	HEX 0	/ Sum is stored here
	ORG 150	/ Origin of operands is HEX 150
	DEC 75	/ First operand
	:	
	:	
	DEC 23	/ Last operand
	END	/ End of symbolic program



TABLE 6-13 Symbolic Program to Add 100 Numbers

Line			
1		ORG 100	/Origin of program is HEX 100
2		LDA ADS	/Load first address of operands
3		STA PTR	/Store in pointer
4		LDA NBR	/Load minus 100
5		STA CTR	/Store in counter
6		CLA	/Clear accumulator
7	LOP,	ADD PTR I	/Add an operand to AC
8		ISZ PTR	/Increment pointer
9		ISZ CTR	/Increment counter
10		BUN LOP	/Repeat loop again
11		STA SUM	/Store sum
12		HLT	/Halt
13	ADS,	HEX 150	/First address of operands
14	PTR,	HEX 0	/This location reserved for a pointer
15	NBR,	DEC -100	/Constant to initialize counter
16	CTR,	HEX 0	/This location reserved for a counter
17	SUM,	HEX 0	/Sum is stored here
18		ORG 150	/Origin of operands is HEX 150
19		DEC 75	/First operand
.			
.			
.			
118		DEC 23	/Last operand
119		END	/End of symbolic program

